Algorithmes et Pensée Computationnelle

Consolidation 1

Les exercices de cette série sont une compilation d'exercices semblables à ceux vus lors des semaines précédentes. Le but de cette séance est de consolider les connaissances acquises lors des travaux pratiques des semaines 1 à 6.

Le code présenté dans les énoncés se trouve sur Moodle, dans le dossier `code`. Les durées indiquées (🕓) sont données à titre strictement indicatif.

⚠ Attention

Il n'est pas nécessaire de faire tous les exercices pendant la séance de TP. Privilégiez les exercices qui vous paraissent difficiles et posez des questions aux assistants.

1 Introduction et architecture des ordinateurs

1.1 Conversion

Question 1: (🕓 *10 minutes*) **Conversion**

1. Convertir le nombre $4321_{(5)}$ en base 10.
2. Convertir le nombre $ABC_{(16)}$ en base 2.
3. Convertir le nombre $11101_{(2)}$ en base 10.

Conseil

Rappelez-vous qu'en hexadécimal, A vaut 10, B vaut 11, C vaut 12, D vaut 13, E vaut 14 et F vaut 15.

Solution

1. $FFFFFF_{(16)} = 16777215_{(10)}$
2. $4321_{(5)} = 586_{(10)}$
3. $ABC_{(16)} = 101010111100_{(2)}$
4. $11101_{(2)} = 29_{(10)}$

1.2 Conversion et arithmétique

Question 2: (🕓 *5 minutes*) **Conversion, addition et soustraction :**
Effectuez les opérations suivantes :

1. $10110101_{(2)} + 00101010_{(2)} = ..._{(10)}$
2. $70_{(10)} - 10101010_{(2)} = ..._{(10)}$

Conseil

Convertissez dans une base commune avant d'effectuer les opérations.

Solution

1. $10110101_{(2)} + 00101010_{(2)} = 223_{(10)}$
2. $70_{(10)} - 10101010_{(2)} = -100_{(10)}$

1.3 Modèle de Von Neumann

Dans cette section, nous allons simuler une opération d'addition dans le **modèle de Von Neumann**, il sera demandé à chaque étape (FDES) de donner la valeur des registres.

État d'origine :
À l'origine, notre `Program Counter (PC)` vaut `00100001`.

Dans la mémoire, les instructions sont les suivantes :

Adresse	Valeur
00100001	00110100
00101100	10100110
01110001	11111110

Les registres sont les suivants :

Registre	Valeur
00	01111111
01	00100000
10	00101101
11	00001100

Les opérations disponibles pour l'unité de contrôle sont les suivantes :

Numéro	Valeur
00	ADD
01	XOR
10	MOV
11	SUB

Question 3: (*5 minutes*) **Fetch**
À la fin de l'opération `FETCH`, quelles sont les valeurs du `Program Counter` et de l'`Instruction Register` ?

Conseil

Pour rappel, l'unité de contrôle (Control Unit) commande et contrôle le fonctionnement du système. Elle est chargée du séquençage des opérations. Après chaque opération FETCH, la valeur du Program Counter est incrémentée (valeur initiale + 1).

Solution

PC = $00100001_{(2)}$ + 1 = $00100010_{(2)}$
IR = $00110100_{(2)}$

Question 4: (*5 minutes*) **Decode**

1. Quelle est la valeur de l'opération à exécuter ?
2. Quelle est le numéro du registre dans lequel le résultat doit être enregistré ?
3. Quelle est la valeur du premier nombre de l'opération ?
4. Quelle est la valeur du deuxième nombre de l'opération ?

Conseil

- Pensez à décomposer la valeur de l'`Instruction Register` pour obtenir toutes les informations demandées.
- Utilisez la même convention que celle présentée dans les diapositives du cours (Architecture des ordinateurs - semaine 1).
- Les données issues de la décomposition de l'`Instruction Register` ne sont pas des valeurs brutes, mais des références. Trouvez les tables concordantes pour y récupérer les valeurs.

Solution

00 : `ADD` (valeur de l'opération à exécuter)
11 : Adresse du registre dans lequel le résultat doit être enregistré
01 : $00100000_{(2)}$ (premier nombre)
00 : $01111111_{(2)}$ (deuxième nombre)

Question 5: (*5 minutes*) **Execute**
Quel est le résultat de l'opération ?

Conseil

Toutes les informations permettant d'effectuer l'opération se trouvent dans les données de l'`Instruction Register`.

Solution

$00100000_{(2)} + 01111111_{(2)} = 10011111_{(2)}$

2 Logiciels système

Question 6: (*10 minutes*) En utilisant l'invite de commandes (Terminal), créer un programme Python demandant à l'utilisateur d'entrer son nom et son prénom. Lorsque le programme est exécuté, la phrase suivante doit s'afficher : *"Bonjour, je m'appelle *prénom nom*"*.

Solution

```
1  # utilisez nano terminal.py afin de créer ce fichier
2  prenom = input("Entrez votre prénom")
3  nom = input("Entrez votre nom")
4  print("Bonjour, je m'appelle " + prenom + " " + nom)
```

Question 7: (*5 minutes*) Sous Linux et MacOS, laquelle de ces commandes modifie le `filesystem` ?

1. `ls -la`
2. `sudo rm -rf ~/nano`
3. `sudo kill -9 3531`
4. `more nano.txt`
5. `cd ..`
6. `pwd`
7. Aucune de ces réponses n'est correcte.

Solution

La commande `sudo rm -rf ~/nano` permet de supprimer le répertoire `nano` situé dans le dossier `/Users/<Utilisateur_courant>` en mode super-utilisateur (utilisateur ayant des droits étendus sur le système).

Attention

Certaines commandes listées ci-dessus peuvent avoir des conséquences irréversibles.
Pour avoir une description détaillée d'une commande, vous pouvez ajouter `man` devant la commande sous Linux/MacOS ou ajouter `-h, --help` ou `/?` après la commande sous Windows.

3 Programmation de base

Question 8: (*10 minutes*)

1. Convertir $52_{(10)}$ en base 2 sur 8 bits.
2. Convertir $100_{(10)}$ en base 2 sur 8 bits.
3. Calculer en base 2 la soustraction : $01100100_{(2)}$ - $00110100_{(2)}$.
4. Déterminer, en complément à deux, l'opposé (l'inverse additif) de $0110000_{(2)}$.

Conseil

- Se référer aux techniques des semaines 1 et 3.
- Faire un tableau des puissances de 2 sur 8 bits.

Solution

1. $52_{(10)} = 32_{(10)} + 16_{(10)} + 4_{(10)} = 00110100_{(2)}$
2. $100_{(10)} = 64_{(10)} + 32_{(10)} + 4_{(10)} = 01100100_{(2)}$
3. $01100100_{(2)}$ - $00110100_{(2)} = 00110000_{(2)}$
4. (a) **Complément à 1 :** not($0110000_{(2)}$) = $1001111_{(2)}$
   (b) **Complément à 2 :** Complément à 1 + 1 = $1001111_{(2)} + 1_{(2)} = 1010000_{(2)}$

Question 9: (*5 minutes*) Qu'affiche le programme suivant ?

```
1  a = 2.0
2  b = 5
3  c = "hello world"
4
5  if type(a) == int:
6      print("output 1")
7  if (type(a) == int or type(c) == str) and b!=2.0:
8      print("output 2")
9  if (a*b >= 10 or type(c) == bool) and type(a) == float:
10     print("output 3")
```

Solution

output 2
output 3

Question 10: (*5 minutes*) À partir du dictionnaire suivant :

```
    jours_semaine = {1:"lundi", 2:"mardi", 3:"mercredi", 4:"jeudi", 5:"jeudi",
    6:"samedi"}

```

Remplacez la valeur associée au nombre 5 par “vendredi”. Ajoutez la paire clé-valeur `7 : "dimanche"`. Affichez la valeur associée à la clé 3. Créez une liste, en compréhension, contenant les valeurs associées aux clés qui sont des nombres pairs.

Solution

```
jours_semaine = {1:"lundi", 2:"mardi", 3:"mercredi", 4:"jeudi",
    5:"jeudi", 6:"samedi"}
jours_semaine[5] = "vendredi"
jours_semaine[7] = "dimanche"
print(jours_semaine[3])
liste_jours = [jours_semaine[x] for x in jours_semaine if x % 2 == 0]
```

4 Fonctions, mémoire et exceptions

Question 11: (*5 minutes*) [Python] **Portée des variables**
Qu’affiche le programme suivant ?

```
x = 2

def fonction():
    x = 3

    def fonction2():
        global x
        print("x=" + str(x))

    fonction2()
    print("x=" + str(x))

print(fonction())
```

Solution

```
x=2
x=3
None
```

Question 12: (*10 minutes*) [Python] **Fonctions et exceptions**
Écrivez une fonction `check_name(name)` qui prend comme paramètre `name` et qui affiche `Bonjour {name}`. Levez une exception de type `ValueError` si le paramètre n’est pas de type `str` ou s’il s’agit d’une chaîne de caractères vide.

Conseil

En Python, `type(variable)` permet de déterminer le type d’une variable.

Solution

Python :

```python
def check_name(nom):
    if type(nom) != str or nom == "":
        raise ValueError("Invalid type of argument")
    else:
        print("Bonjour", nom)
```

Question 13: (*5 minutes*) [Java] **Gestion d'exceptions**

En Java, quel mot-clé est utilisé pour lever une exception ?

1. `catch`
2. `throw`
3. `raise`
4. `try`

Solution

Le mot-clé pour lever une exception en Java est `throw`.

Question 14: (*5 minutes*) [Java] **Gestion d'exceptions**
Qu'affiche le programme suivant ?

```java
public class Question14 {
  public static void main(String args[]) {
    try {
      int i = 7 / 0;
      System.out.print("OUTPUT 1");
    } catch (ArithmeticException e) {
      System.out.print("OUTPUT 2");
    }
  }
}
```

Solution

Le programme affiche OUTPUT 2.

5 Itération et récursivité

Question 15: (*10 minutes*) **Fonctions et itération**
En Python, définissez une fonction nommée `overlap` qui prend comme arguments deux listes et retourne une liste des éléments contenus dans les deux listes sans duplication. Par exemple : `print(overlap([1, 2, 3, 4], [9, 4, 3, 7, 1]))` renverra `[1, 3, 4]`.

Solution

```python
def overlap(list1,list2):
    res=[]
    for elm in list1:
        if elm in list2:
            if not(elm in res):
```

```
                res.append(elm)
    return res

print(overlap([1, 2, 3, 4], [9, 4, 3, 7, 1]))
```

Question 16: (*15 minutes*) **Itération et récursivité**
Créez une fonction itérative, puis une fonction récursive qui calculent le nombre de voyelles présentes dans un texte donné.

Conseil

- Pour la version itérative, parcourez toute la chaîne de caractères et incrémentez un compteur lorsque vous avez une voyelle.
- Pour la version récursive, diminuez systématiquement la taille de votre chaîne de caractères. Si l'élément actuel est une voyelle, ajoutez 1, sinon, ajoutez 0.
- Aidez-vous d'une liste contenant toutes les voyelles et de l'instruction `in` en `Python` (`List.contains()` en `Java`).
- En `Python`, pour accéder à un caractère dans une chaîne de caractères, vous pouvez utiliser l'indexation (par exemple : `chaine[0]` pour accéder au premier caractère de la chaîne de caractères `chaine`). En `Java`, vous pouvez utiliser la méthode `charAt()` de la classe `String` (par exemple : `chaine.charAt(0)` pour accéder au premier caractère de la chaîne de caractères `chaine`).

Voici les templates que vous pouvez réutiliser. Ils se trouvent également sur Moodle dans le dossier `code`.

Python

```python
def nb_voyelles_iterative(text) :
    voyelles = ['a','e','i','o','u','y']
    #TODO

def nb_voyelles_recursive(text) :
    voyelles = ['a','e','i','o','u','y']
    #TODO

texte = "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean
    molestie elit ipsum, a tincidunt urna aliquet"

print(nb_voyelles_iterative(texte))
print(nb_voyelles_recursive(texte))
```

Java

```java
import java.util.List;

public class Question16 {

    public static int nb_voyelles_iterative(String S, List L) {
        // TODO
    }

    public static int nb_voyelles_recursive(String S, List L) {
        // TODO
    }

    public static void main(String[] args) {
```

```
        List voyelles = List.of('a', 'e', 'i', 'o', 'u', 'y');
        String texte = "Lorem ipsum dolor sit amet, consectetur adipiscing
    elit. Aenean molestie elit ipsum, a tincidunt urna aliquet";

        System.out.println(nb_voyelles_iterative(texte, voyelles));
        System.out.println(nb_voyelles_recursive(texte, voyelles));
    }
}
```

Solution

Python :

```
def nb_voyelles_iterative(text) :
    c = 0
    voyelles = ['a','e','i','o','u','y']

    for i in text:
        if i in voyelles:
            c += 1
    return c

def nb_voyelles_recursive(text) :
    voyelles = ['a','e','i','o','u','y']

    if len(text) == 0:
        return 0
    else:
        if text[0] in voyelles:
            return 1 + nb_voyelles_recursive(text[1:])
        else:
            return 0 + nb_voyelles_recursive(text[1:])

texte = "Lorem ipsum dolor sit amet, consectetur adipiscing elit.
    Aenean molestie elit ipsum, a tincidunt urna aliquet"

print(nb_voyelles_iterative(texte))
print(nb_voyelles_recursive(texte))
```

Solution

Java :

```
import java.util.List;

public class IterationRecursionSolution {

    public static int nb_voyelles_iterative(String text) {
        List<Character> voyelles = List.of('a', 'e', 'i', 'o', 'u',
    'y');
        int c = 0;
        for (int i = 0; i < text.length(); i++) {
            if (voyelles.contains(text.charAt(i))) {
                c++;
            }
        }
        return c;
    }

    public static int nb_voyelles_recursive(String text) {
```

```
17          List<Character> voyelles = List.of('a', 'e', 'i', 'o', 'u',
        'y');
18          if (text.isEmpty()) {
19              return 0;
20          } else {
21              if (voyelles.contains(text.charAt(0))) {
22                  return 1 + nb_voyelles_recursive(text.substring(1,
        text.length()));
23              } else {
24                  return 0 + nb_voyelles_recursive(text.substring(1,
        text.length()));
25              }
26          }
27      }
28
29      public static void main(String[] args) {
30          String text = "Lorem ipsum dolor sit amet, consectetur
        adipiscing elit. Aenean molestie elit ipsum, a tincidunt urna
        aliquet";
31
32          System.out.println(nb_voyelles_iterative(text));
33          System.out.println(nb_voyelles_recursive(text));
34      }
35  }
```

Question 17: (*5 minutes*) Lecture de code (Récursivité)
Qu'affichent les programmes suivants ?

Conseil

Ces deux programmes comportent des fonctions récursives, lisez bien le code de haut en bas et lorsque la fonction fait appel à elle-même, revenez au début de la fonction et effectuez de nouveau les instructions avec les nouveaux paramètres. Une feuille de papier pourrait vous être utile !

Programme 1 :

```
1  def recursion_1(S) :
2      if len(S) == 1 :
3          return S[0]
4      else :
5          return S[0] + recursion_1(S[1:]) + S[0]
6
7  print(recursion_1("Python"))
```

Programme 2 :

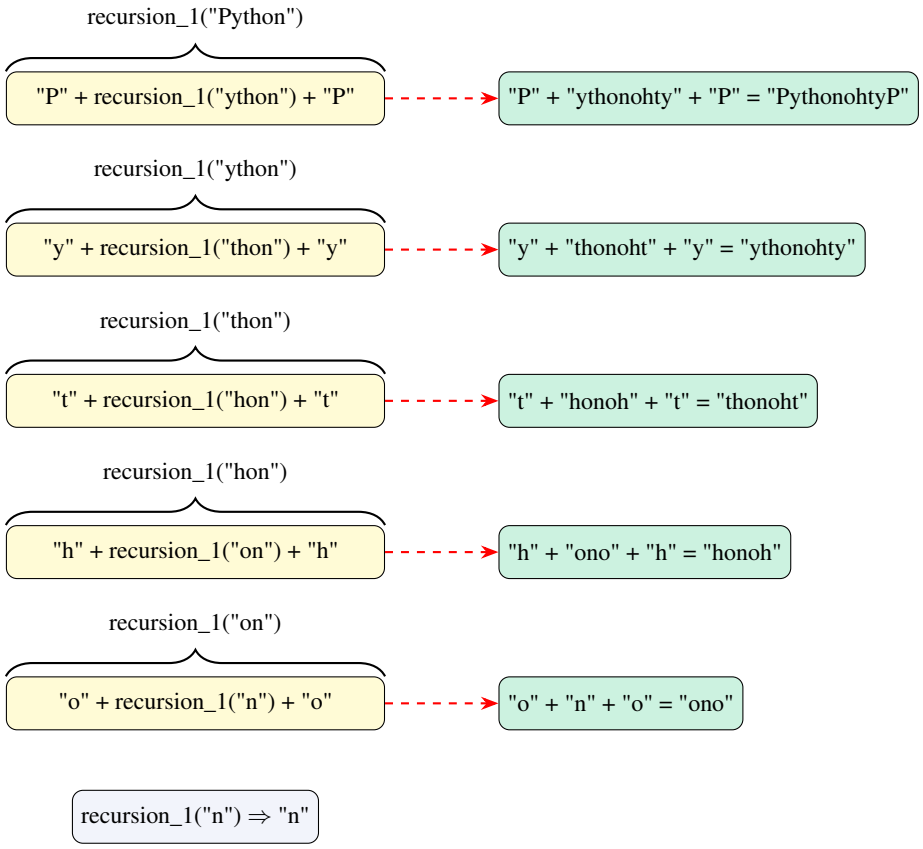
```
1  def recursion_2(L) :
2      if len(L) == 1 :
3          print(L[0])
4      else :
5          recursion_2(L[1:])
6          print(L[0])
7          recursion_2(L[1:])
8
9  Liste = ["J","adore","Python"]
10 recursion_2(Liste)
```

Solution

Programme 1 :

PythonohtyP

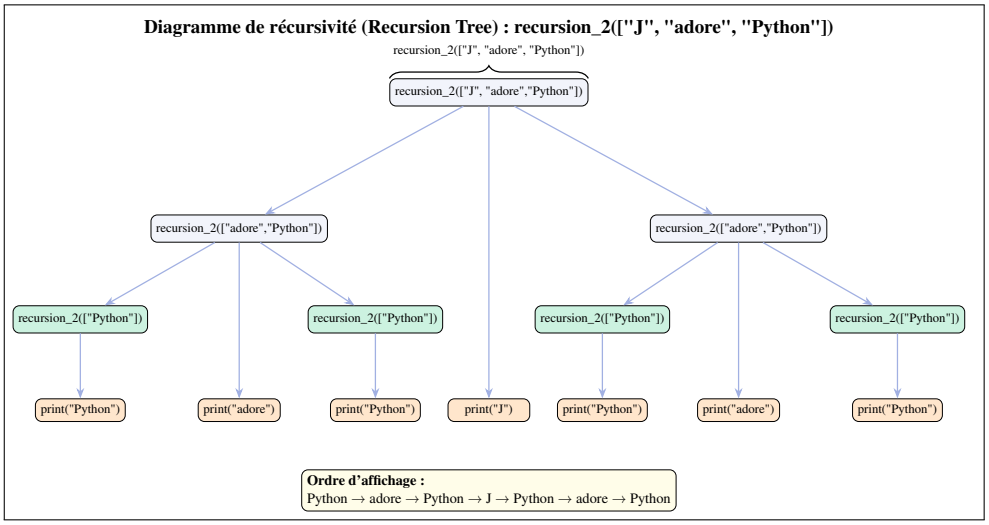
Diagramme de récursivité (Recursion Tree) : recursion_1("Python")



Solution

Programme 2 :

Python
adore
Python
J
Python
adore
Python



Question 18: (*15 minutes*) **Itération et récursivité**

Demandez à l'utilisateur le nombre de lignes et de colonnes d'une grille, puis la ligne et la colonne d'une case à sélectionner (numérotées à partir de 1). Avec des boucles imbriquées, affichez la grille bordée comme ci-dessous : chaque case contient `-`, sauf la case sélectionnée qui contient `X`.

Exemple pour 2 lignes, 3 colonnes et la case de ligne 1, colonne 2 :

```
|- X - |
|- - - |
```

Conseil

Parcourez les lignes avec une boucle et les colonnes avec une autre. Une condition repère la case sélectionnée. Utilisez `print(..., end="")` pour rester sur la même ligne et `print()` pour passer à la suivante.

Solution

```
1  sizeX = int(input("Définissez le nombre de lignes que le tableau va
       faire :"))
2  sizeY = int(input("Définissez le nombre de colonnes que le tableau va
       faire :"))
3
4  # On va sélectionner une case en rentrant ces coordonnées
5  valueX = int(input("Entrez une line à sélectionner : "))
6  valueY = int(input("Entrez une case à sélectionner : "))
7
8  # Iterate over the lines
9  for x in range(0,sizeX):
10     if x == (valueX-1) and valueY == 0 :
11         print("")
12         print("X", end = '')
13     else:
14         print("")
15         print("|", end = '')
16
17     #print the column
18     for y in range (0,sizeY):
19         if x == (valueX-1) and y == (valueY-1):
20             print("X ", end='')
```

```
            #print the basic element
        else:
            print("- ", end = '')

    #Iterate over the columns
    print("|")
```

6 Programmation orientée objet - classes et héritage

Question 19: (*15 minutes*) **Jeu de rôle**
Dans cet exercice, vous allez mettre en place un simple jeu de rôle. Le concept d'héritage sera très utile dans ce jeu de rôle étant donné que les différentes classes de personnages possèdent certains attributs ou actions similaires. Les personnages de notre jeu sont le Guerrier, le Paladin, le Magicien et le Chasseur.
Ainsi, il serait judicieux de créer une première classe `Personnage`. Un personnage est un objet qui possède plusieurs attributs :

- `nom` : le nom du personnage de type chaîne de caractères
- `niveau` : le niveau du personnage de type entier
- `pv` : les points de vie du personnage de type entier
- `vitalite` : la vitalité (ou santé) du personnage de type entier
- `force` : la force du personnage de type entier
- `dexterite` : la dextérité du personnage de type entier
- `endurance` : l'endurance du personnage de type entier
- `intelligence` : l'intelligence du personnage de type entier

Afin de compléter le programme.

- Définissez le constructeur de Personnage, qui prend en paramètre les valeurs de tous les attributs ci-dessus, ainsi que les getters.
- Redéfinissez `toString()` pour renvoyer tous les attributs sur une seule ligne, sous la forme `nom de l'attribut: valeur`, séparés par des virgules.
- Chaque personnage de ce jeu a un compteur (points) de vies stocké dans l'attribut `pv`. Celui-ci va être modifié par un `setter`. Définissez son `setter`.
- Créer les classes `Guerrier, Paladin, Magicien et Chasseur` qui héritent de `Personnage` et définissez leurs constructeurs.

```
public abstract class Personnage {

    private ...
    ...

    public Personnage(...){
        this.nom = ...
        ...

    }

    @Override
    public String toString() {
        ...
    }
    public abstract void attaqueBasique(Personnage autre);
    ...
    ...
}
```

- Dans chacune de ces classes, implémentez la méthode `attaqueBasique(Personnage cible)`, qui ne renvoie aucune valeur. Les dégâts dépendent des attributs de l'attaquant :
  - `Guerrier` : inflige des dégâts égaux à 50 % de sa `force`.
  - `Paladin` : inflige des dégâts égaux à 70 % de son `endurance`.
  - `Magicien` : inflige des dégâts égaux à 50 % de son `intelligence`.
  - `Chasseur` : inflige des dégâts égaux à 50 % de son `endurance`.

  Convertissez les dégâts en entier avec `(int)` (la partie décimale est supprimée). Soustrayez-les aux `pv` actuels de la cible via `setPv()`, sans descendre sous zéro. Affichez ensuite une ligne indiquant qui attaque qui, puis une ligne donnant le nom de la cible et ses `pv` restants.

```
1
2  // Placez chaque classe dans un fichier portant son nom.
3  public class Guerrier extends Personnage {
4
5  }
6
7  public class Magicien extends Personnage {
8
9  }
10
11 public class Paladin extends Personnage {
12
13 }
14
15 public class Chasseur extends Personnage {
16
17 }
```

Solution

```
1  public abstract class Personnage {
2
3      private String nom;
4      private int niveau;
5      private int pv;
6
7      private int vitalite;
8      private int force;
9      private int dexterite;
10     private int endurance;
11     private int intelligence;
12
13
14     public Personnage(String nom, int niveau, int pv, int vitalite,
15                       int force, int dexterite, int endurance, int
       intelligence) {
16         this.nom = nom;
17         this.niveau = niveau;
18         this.pv = pv;
19
20         this.vitalite = vitalite;
21         this.force = force;
22         this.dexterite = dexterite;
23         this.endurance = endurance;
24         this.intelligence = intelligence;
25
26     }
27
28     @Override
29     public String toString() {
30         return "Nom : " + nom + ", niveau : " + niveau
```

```
31              + ", PV : " + pv + ", vitalite : " + vitalite
32              + ", force : " + force + ", dexterite : " + dexterite
33              + ", endurance : " + endurance
34              + ", intelligence : " + intelligence;
35      }
36
37
38      public abstract void attaqueBasique(Personnage autre);
39
40      public String getNom() {
41          return nom;
42      }
43
44      public int getIntelligence() {
45          return intelligence;
46      }
47
48      public int getNiveau() {
49          return niveau;
50      }
51
52      public int getPv() {
53          return pv;
54      }
55
56      public int getVitalite() {
57          return vitalite;
58      }
```

Solution

```
60      public int getForce() {
61          return force;
62      }
63
64      public int getDexterite() {
65          return dexterite;
66      }
67
68      public int getEndurance() {
69          return endurance;
70      }
71
72      public void setPv(int pv) {
73          if (pv > 0) {
74              this.pv = pv;
75          }
76          else {
77              this.pv = 0;
78          }
79      }
80  }
```

Solution

```
1   public class Guerrier extends Personnage {
2       public Guerrier(String nom, int niveau, int pv, int vitalite,
3                       int force, int dexterite, int endurance, int
```

```
        intelligence) {
4           super(nom, niveau, pv, vitalite, force, dexterite, endurance,
        intelligence);
5       }
6
7       @Override
8       public void attaqueBasique(Personnage autre) {
9           int degats = (int) (getForce() * 0.5);
10          autre.setPv(autre.getPv() - degats);
11
12          System.out.println(getNom() + " attaque " + autre.getNom());
13          System.out.println(autre.getNom() + " : " + autre.getPv() + "
        PV restants");
14      }
15  }
```

Solution

```
1   public class Magicien extends Personnage {
2       public Magicien(String nom, int niveau, int pv, int vitalite,
3                       int force, int dexterite, int endurance, int
        intelligence) {
4           super(nom, niveau, pv, vitalite, force, dexterite, endurance,
        intelligence);
5       }
6
7       @Override
8       public void attaqueBasique(Personnage autre) {
9           int degats = (int) (getIntelligence() * 0.5);
10          autre.setPv(autre.getPv() - degats);
11
12          System.out.println(getNom() + " attaque " + autre.getNom());
13          System.out.println(autre.getNom() + " : " + autre.getPv() + "
        PV restants");
14      }
15  }
```

Solution

```
1   public class Paladin extends Personnage {
2       public Paladin(String nom, int niveau, int pv, int vitalite,
3                      int force, int dexterite, int endurance, int
        intelligence) {
4           super(nom, niveau, pv, vitalite, force, dexterite, endurance,
        intelligence);
5       }
6
7       @Override
8       public void attaqueBasique(Personnage autre) {
9           int degats = (int) (getEndurance() * 0.7);
10          autre.setPv(autre.getPv() - degats);
11
12          System.out.println(getNom() + " attaque " + autre.getNom());
13          System.out.println(autre.getNom() + " : " + autre.getPv() + "
        PV restants");
14      }
15  }
```

Solution

```java
public class Chasseur extends Personnage {
    public Chasseur(String nom, int niveau, int pv, int vitalite,
                    int force, int dexterite, int endurance, int
    intelligence) {
        super(nom, niveau, pv, vitalite, force, dexterite, endurance,
    intelligence);
    }

    @Override
    public void attaqueBasique(Personnage autre) {
        int degats = (int) (getEndurance() * 0.5);
        autre.setPv(autre.getPv() - degats);

        System.out.println(getNom() + " attaque " + autre.getNom());
        System.out.println(autre.getNom() + " : " + autre.getPv() + "
    PV restants");
    }
}
```

7 Quiz général

7.1 Python

Question 20: (🕓 *2 minutes*)
En Python, `'Hello'` équivaut à `"Hello"`.

1. - Vrai
2. - Faux

Solution

Vrai : En Python, les doubles guillemets et les guillemets sont équivalents.

Question 21: (🕓 *2 minutes*)
Dans une fonction, nous pouvons utiliser les instructions `print()` ou `return`, elles ont le même rôle.

1. - Vrai
2. - Faux

Solution

Faux
`print()` permet uniquement d'afficher un message dans la console. Autrement dit, `print()` sert à communiquer un message à l'utilisateur final du programme, celui-ci n'ayant pas accès au code.

`return` est une instruction qu'on utilise à l'intérieur d'une fonction pour renvoyer le résultat de la fonction lorsqu'elle a été appelée. Exemple : la fonction `len(L)` renvoie la longueur de la liste L. Il est important de noter que dans une fonction, toutes les instructions définies après un `return` ne seront pas exécutées.

Admettons que nous ayons une fonction qui renvoie le double d'un nombre. Dans notre programme, nous souhaitons ajouter le résultat de cette fonction à un nombre quelconque.

Solution

En Python, notre programme ressemblera à ça :

```
1  def double_nombre(nombre):
2      return nombre * 2
3
4  nombre1 = 2
5  resultat = nombre1 + double_nombre(2)
6  print(resultat)
```

Notre programme renverra une erreur de type `TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'` si on remplace la ligne 2 par `print(nombre**2)` car la fonction `double_nombre` renverra `None`.

Question 22: (*2 minutes*)
Lorsqu'on fait appel à une fonction, les arguments doivent nécessairement avoir les mêmes noms que les paramètres définis dans la fonction. Exemple :

```
1  def recherche_lineaire(Liste, x):
2      for i in Liste:
3          if i == x:
4              return x in Liste
5      return -1
6
7  Liste = [1,3,5,7,9]
8  x = 3
9
10 recherche_lineaire(Liste,x)
```

1. - Vrai
2. - Faux

Solution

Faux
Le nom des variables données en argument n'a aucune importance tant que le type de variable est respecté. Dans notre exemple, la fonction s'attend à recevoir une **liste** comme premier argument et un **entier** comme deuxième argument. Ici, nous aurions pu nommer la liste `nbr_impair` et x `valeur` et appeler la fonction `recherche_lineaire(nbr_impair, valeur)`

Question 23: (*2 minutes*)
En Python, il est possible de faire appel à une fonction définie "plus bas" dans le code sans que cela ne pose problème.

```
1  import math
2
3  nombre_decimal_pi(4)
4
5  def nombre_decimal_pi(int):
6      return round(math.pi,int)
```

1. - Vrai
2. - Faux

Solution

Faux : À l'exception des fonctions intégrées (il s'agit des fonctions déjà intégrées au langage Python telles que `print(), len(), abs(), etc...`). Dans les langages suivant une logique

de programmation impérative (c'est le cas de Java et Python), une fonction doit nécessairement être définie **avant** d'être appelée. https://fr.wikipedia.org/wiki/Programmation_imp%C3%A9rative

Question 24: (*5 minutes*)
Les trois fonctions suivantes renvoient-elles systématiquement des résultats identiques ? Les fonctions sont censées retourner le nombre `pi` avec le nombre de décimales (au moins une et au maximum 15) indiqué en paramètre.

```python
import math

def nombre_decimal_pi(arg):
    if arg > 15:
        arg = 15
    elif arg < 0:
        arg = 1
    resultat = round(math.pi,arg)
    return resultat

print(nombre_decimal_pi(-2))
print(nombre_decimal_pi(4))
print(nombre_decimal_pi(20))
```

```python
import math
def nombre_decimal_pi(arg):
    if arg > 15:
        resultat = round(math.pi,15)
    elif arg < 0:
        resultat = round(math.pi,1)
    else:
        resultat = round(math.pi,arg)
    return resultat

print(nombre_decimal_pi(-2))
print(nombre_decimal_pi(4))
print(nombre_decimal_pi(20))
```

```python
import math

def nombre_decimal_pi(arg):
    if arg > 15:
        return round(math.pi,15)
    elif arg < 0:
        return round(math.pi,1)
    else:
        return round(math.pi,arg)

print(nombre_decimal_pi(-2))
print(nombre_decimal_pi(4))
print(nombre_decimal_pi(20))
```

1. - Vrai
2. - Faux

Solution

Vrai : Les trois fonctions produisent des résultats identiques. Si besoin, exécutez le code dans IntelliJ.

7.2 Java

Question 25: (*3 minutes*)
Observez les deux programmes suivants en Java. Lequel a-t-il une bonne structure et peut être compilé sans erreur ?

```java
//Programme A
public class Main {
    public static void main(String[] args) {
```

```
        ma_function();
        autre_fonction();

        static void ma_function(){
            System.out.println("Voici ma fonction!");
        }

        static void autre_fonction(){
            System.out.println("Une autre fonction!");
        }
    }
}
```

```
//Programme B
public class Main {

    public static void main(String[] args) {
    ma_function();
    autre_fonction();
    }

    static void ma_function(){
        System.out.println("Voici ma fonction!");
    }

    static void autre_fonction(){
        System.out.println("Une autre fonction!");
    }
}
```

1. Programme A
2. Programme B

Solution

Le programme B
Le fichier dans son ensemble représente une classe Java, ici la classe s'appelle **Main**. À l'intérieur de cette classe se trouve la méthode `public static void main()`, il s'agit de la porte d'entrée de notre programme, celle que l'on exécute et celle dans laquelle nous rédigeons notre code.
Les autres fonctions, qui peuvent être appelées, se définissent au sein de la classe au même échelon que la fonction `public static void main()` comme dans le programme B ci-dessus.

Question 26: (*2 minutes*) Exercice 1
L'indentation des lignes de code en Java est aussi importante qu'en Python.

1. - Vrai
2. - Faux

Solution

Faux
En Java, le compilateur ne prend pas en compte l'indentation pour interpréter le programme, il comprend la structure à l'aide des parenthèses, des accolades et encore des point-virgules qui indiquent la fin d'une instruction. Toutefois, l'indentation est un aspect important de la programmation car elle sert à bien structurer visuellement votre code.

En Python, l'indentation définit la structure de votre code. Elle est donc indispensable

pour la bonne interprétation et la bonne exécution de votre programme.